Phase 4 — Deep Learning

ExamGuard AI — Implementation Guide

PyTorch or TensorFlow — Your Deep Learning Engine

What Is This?

You know Scikit-learn from Phase 3? It was great for traditional ML — decision trees, random forests, logistic regression. But now we need neural networks — the brain-like models that power image recognition, voice assistants, and self-driving cars.

PyTorch and TensorFlow are the tools that let you build neural networks.

Think of it this way:

- Scikit-learn = a regular calculator (handles basic math well)
- PyTorch/TensorFlow = a scientific graphing calculator (handles complex, layered computations)

Both do the same job. Both are free. Both are used by top companies. You just need to pick one.

PyTorch vs TensorFlow — Quick Comparison

Feature	PyTorch	TensorFlow
Made by	Meta (Facebook)	Google
Learning curve	Easier for beginners	Steeper initially
Code style	Feels like regular Python	Has its own way of doing things
Debugging	Easy — standard Python debugger works	Harder — runs in "graph mode"
Used by	Most researchers, universities	Production systems, Google products
Community	Growing fast, dominant in research	Huge, lots of tutorials
GPU support	Built-in, easy	Built-in, easy
Job market	Increasingly demanded	Still widely used in industry

My Recommendation: Start with PyTorch

Why?

- 1. Reads like Python — if you know Python, PyTorch code makes sense immediately
- 2. Easier debugging — when something breaks, you can figure out why
- 3. Research standard — most new AI papers use PyTorch, so latest techniques appear here first
- 4. ExamGuard models — YOLO (our object detection model) is built on PyTorch

You can always learn TensorFlow later. The concepts transfer directly.

WHY This Matters for ExamGuard

Every deep learning component in ExamGuard runs on one of these frameworks:

ExamGuard Component	What It Does	Framework Needed
CNN for behavior classification	Looks at frames, says "cheating" or "normal"	PyTorch
YOLO for object detection	Finds phones, chits, earpieces in	PyTorch

	frame	
Face recognition	Identifies which student is which	PyTorch
Pose estimation	Tracks body position and movement	PyTorch/TF
Gaze estimation	Detects where eyes are looking	PyTorch

Without PyTorch or TensorFlow, you literally cannot build ExamGuard. This is the engine that powers everything from Phase 4 onward.

Real ExamGuard Connection

Here is what happens when a camera captures a frame in your exam hall:

```
Camera captures frame
      |
      v
[OpenCV reads the frame]        <-- Phase 5
      |
      v
[PyTorch CNN processes it]      <-- THIS IS WHERE PYTORCH COMES IN
      |
      v
[Model outputs: "cheating" 87%]  <-- PyTorch calculates this
      |
      v
[Alert sent to supervisor]      <-- Phase 7
```

Every single model prediction — "Is this student cheating?", "Is that a phone?", "Where are they looking?" — is a PyTorch computation happening in milliseconds.

What You Need to Learn

1. Tensors — The Building Block

Everything in deep learning is a tensor (basically a multi-dimensional array).

```
import torch

# A single number (0D tensor)
x = torch.tensor(5)

# A list of numbers (1D tensor) — like exam scores
scores = torch.tensor([85, 92, 78, 95])

# A 2D tensor — like a grayscale image (rows x columns of pixel values)
image = torch.tensor([[120, 130, 125],
                      [118, 135, 128],
                      [122, 131, 127]])

# A 3D tensor — like a color image (3 channels x height x width)
color_image = torch.randn(3, 224, 224)  # RGB, 224x224 pixels
```

ExamGuard connection: Every camera frame becomes a tensor before any model can process it.

2. Building Layers — The Model Architecture

A neural network is layers stacked on top of each other:

```
import torch.nn as nn

# A simple neural network
model = nn.Sequential(
    nn.Linear(784, 128),    # Input layer: 784 pixels → 128 neurons
    nn.ReLU(),              # Activation: "turn on" important neurons
    nn.Linear(128, 64),     # Hidden layer: 128 → 64 neurons
    nn.ReLU(),
    nn.Linear(64, 2)        # Output: 64 → 2 classes (cheating/normal)
)
```

3. Forward Pass — Making a Prediction

```
# Feed an image through the model
input_image = torch.randn(1, 784)   # One flattened image
output = model(input_image)         # Forward pass
print(output)                       # tensor([[ 0.85, -0.32]])
# First number high → "cheating", Second number high → "normal"
```

4. Loss Functions — How Wrong Was the Prediction?

```
loss_fn = nn.CrossEntropyLoss()

prediction = model(input_image)
actual_label = torch.tensor([0])     # 0 = cheating, 1 = normal

loss = loss_fn(prediction, actual_label)
print(f"Error: {loss.item():.4f}")   # Lower = better
```

5. Optimizers — How the Model Learns

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop (simplified)
for epoch in range(100):
    prediction = model(input_image)
    loss = loss_fn(prediction, actual_label)

    optimizer.zero_grad()   # Reset gradients
    loss.backward()         # Calculate how to improve
    optimizer.step()        # Actually improve the model
```

6. GPU Acceleration

```
# Move model and data to GPU for 10-50x speed boost
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
input_image = input_image.to(device)
```

ExamGuard connection: Processing 120 frames/second requires GPU acceleration.

The Complete Training Flow

1. PREPARE DATA
   Images of cheating/normal → Convert to tensors → Split train/test

2. BUILD MODEL
  Define layers → Choose activation functions

3. TRAIN
  Feed images → Model predicts → Calculate error → Adjust weights → Repeat 1000x

4. EVALUATE
  Test on unseen images → Check accuracy → Good enough? → Deploy

5. DEPLOY
  Save model → Load in ExamGuard → Process live camera frames

Mini Project: Handwritten Digit Classifier (MNIST)

Goal: Build your first neural network that recognizes handwritten digits (0-9).

Why this project? It is the "Hello World" of deep learning. Simple enough to learn on, complex enough to teach real concepts.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Step 1: Load the MNIST dataset (60,000 training images of digits)
transform = transforms.Compose([
   transforms.ToTensor(),
   transforms.Normalize((0.5,), (0.5,))
])

train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_data = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

# Step 2: Build the neural network
class DigitClassifier(nn.Module):
   def __init__(self):
      super().__init__()
      self.network = nn.Sequential(
         nn.Flatten(),            # 28x28 image → 784 numbers
         nn.Linear(784, 256),      # First hidden layer
         nn.ReLU(),
         nn.Linear(256, 128),      # Second hidden layer
         nn.ReLU(),
         nn.Linear(128, 10)        # Output: 10 digits (0-9)
      )

   def forward(self, x):
      return self.network(x)
```

```
model = DigitClassifier()

# Step 3: Set up training
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Step 4: Train the model
for epoch in range(5):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for images, labels in train_loader:
        # Forward pass
        predictions = model(images)
        loss = loss_fn(predictions, labels)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        correct += (predictions.argmax(1) == labels).sum().item()
        total += labels.size(0)

    accuracy = 100 * correct / total
    print(f"Epoch {epoch+1}/5 | Loss: {total_loss:.2f} | Accuracy: {accuracy:.1f}%")

# Step 5: Test on unseen data
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        predictions = model(images)
        correct += (predictions.argmax(1) == labels).sum().item()
        total += labels.size(0)

print(f"\nTest Accuracy: {100 * correct / total:.1f}%")
# Expected: ~97% accuracy!

# Step 6: Save the model (just like you will save ExamGuard models)
torch.save(model.state_dict(), 'digit_classifier.pth')
print("Model saved!")
```

What You Will Learn From This Project:

- How to load and prepare image datasets

- How to build a neural network from scratch
- The complete training loop (forward → loss → backward → update)
- How to evaluate accuracy on test data
- How to save a trained model

Connection to ExamGuard:

Replace "digits 0-9" with "cheating vs normal" and you have the same workflow ExamGuard uses. Same code structure, different data, different labels.

Key Takeaway

PyTorch is not just a library — it is the foundation that every ExamGuard AI model is built on. Every frame analysis, every detection, every classification runs through PyTorch. Master the basics here, and everything in Phases 4 and 5 will click into place.

CNN — Convolutional Neural Network (THE Image Model)

What Is This?

A Convolutional Neural Network (CNN) is a special type of neural network designed specifically to understand images. While a regular neural network sees an image as a flat list of numbers, a CNN sees it as a 2D picture — just like your eyes do.

Think of it this way:

- Regular neural network = reading a book with all the words in one long line (confusing)
- CNN = reading a book with proper paragraphs and pages (makes sense)

CNNs are the standard model for any task involving images. If you are working with pictures or video, you are using a CNN.

WHY This Is CORE to ExamGuard

This is not optional. This is not "nice to have." CNN is the HEART of ExamGuard.

Every single camera frame in your exam monitoring system goes through a CNN. Here is what happens:

```
Camera captures frame of exam hall
        |
        v
 CNN Layer 1: Detect edges (lines, curves)
        |
        v
 CNN Layer 2: Detect shapes (circles, rectangles)
        |
        v
 CNN Layer 3: Detect body parts (hands, heads, shoulders)
        |
        v
 CNN Layer 4: Detect behaviors (looking sideways, hand extended)
        |
        v
 OUTPUT: "Student looking at neighbor's paper" → CHEATING FLAG
```

Without CNN, ExamGuard is blind. It cannot see anything in the camera feeds.

How CNN Works — Step by Step

The Big Picture

```
Input Image (224x224 pixels)
    |
    v
[Convolution Layers] → Find patterns (edges, textures, shapes)
    |
    v
[Pooling Layers] → Shrink the data (keep important stuff, throw away noise)
    |
    v
[Fully Connected Layers] → Make the final decision
```

```
  |
  v
Output: "cheating" (87%) or "normal" (13%)
```

Step 1: Convolution — Pattern Detection

A convolution is like sliding a magnifying glass over the image, looking for specific patterns.

```
Image (a small 5x5 section):        Filter (3x3, looking for vertical edge):
1 1 1 0 0                           1  0 -1
1 1 1 0 0                           1  0 -1
1 1 1 0 0                           1  0 -1
1 1 1 0 0
1 1 1 0 0
```

```
The filter slides across the image, multiplying and adding at each position.
Where it finds a vertical edge → HIGH number (strong match)
Where there is no edge → LOW number (weak match)
```

ExamGuard connection: Early filters detect edges of bodies, phones, papers. Later filters detect complex shapes like "hand holding a phone" or "head turned sideways."

Step 2: Activation (ReLU) — Keep the Good Stuff

After convolution, we apply ReLU: if the value is negative, make it 0. If positive, keep it.

```
Before ReLU: [-2, 5, -1, 3, 0, -4, 8]
After ReLU:  [ 0, 5,  0, 3, 0,  0, 8]
```

This helps the network focus on what IS there, not what is NOT there.

Step 3: Pooling — Shrink Without Losing Meaning

Max Pooling takes a small area and keeps only the highest value.

```
Before pooling (4x4):    After MaxPool 2x2 (2x2):
6 8 2 4                    8 4
3 1 5 7    →               3 9
2 3 1 9
4 0 6 2
```

Why shrink? A 1080p frame is 1920x1080 = 2 million pixels. Without pooling, the model would be impossibly slow. Pooling keeps the important features while making computation manageable.

ExamGuard connection: We need to process 30+ frames per second. Pooling makes the CNN fast enough for real-time monitoring.

Step 4: Fully Connected — Make the Decision

After convolution and pooling extract features, the fully connected layers act like a traditional classifier:

```
Features extracted by CNN: [head_turned: 0.9, hand_extended: 0.7, eyes_sideways: 0.8]
      |
      v
Fully Connected Layer → Combines all features
      |
      v
Output: cheating = 0.87, normal = 0.13
```

Key Numbers You Need to Know

Filters (Number of Feature Detectors)

```
nn.Conv2d(in_channels=3, out_channels=32, ...)
#                         ^^
#                  32 different patterns to look for
```

- Layer 1: 32 filters (basic edges, colors)
- Layer 2: 64 filters (shapes, textures)
- Layer 3: 128 filters (complex patterns)

Kernel Size (How Big the Magnifying Glass Is)

```
nn.Conv2d(..., kernel_size=3)  # 3x3 filter — most common, good for details
nn.Conv2d(..., kernel_size=5)  # 5x5 filter — bigger patterns, less common
nn.Conv2d(..., kernel_size=7)  # 7x7 filter — only in first layer sometimes
```

Rule of thumb: Use 3x3 for almost everything. It is the standard.

Stride (How Far the Filter Moves Each Step)

```
nn.Conv2d(..., stride=1)  # Moves 1 pixel at a time — detailed but slow
nn.Conv2d(..., stride=2)  # Moves 2 pixels at a time — faster but less detail
```

Padding (Handle the Edges)

```
nn.Conv2d(..., padding=1)  # Adds border of zeros so output size = input size
nn.Conv2d(..., padding=0)  # No border, output shrinks each layer
```

Rule of thumb: Use `padding=1` with `kernel_size=3` to keep dimensions clean.

ExamGuard CNN Architecture

Here is what a real ExamGuard behavior classifier CNN might look like:

```
INPUT: Exam frame (3 x 224 x 224) — color image, 224x224 pixels

Conv Block 1: 3 → 32 filters, 3x3   → ReLU → MaxPool   Output: 32 x 112 x 112
Conv Block 2: 32 → 64 filters, 3x3  → ReLU → MaxPool   Output: 64 x 56 x 56
Conv Block 3: 64 → 128 filters, 3x3 → ReLU → MaxPool   Output: 128 x 28 x 28
Conv Block 4: 128 → 256 filters, 3x3 → ReLU → MaxPool  Output: 256 x 14 x 14

Flatten: 256 x 14 x 14 = 50,176 numbers

Fully Connected 1: 50,176 → 512  → ReLU → Dropout(0.5)
Fully Connected 2: 512 → 2       → Softmax

OUTPUT: [cheating: 0.87, normal: 0.13]
```

Building a CNN in PyTorch

```
import torch
import torch.nn as nn

class ExamBehaviorCNN(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        # Feature extraction layers (find patterns in images)
        self.features = nn.Sequential(
            # Block 1: Find edges and basic patterns
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),         # 224x224 → 112x112

            # Block 2: Find shapes
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),         # 112x112 → 56x56

            # Block 3: Find body parts and objects
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),         # 56x56 → 28x28
        )

        # Classification layers (make the decision)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 512),
            nn.ReLU(),
            nn.Dropout(0.5),            # Prevent overfitting
            nn.Linear(512, 2)           # 2 classes: cheating, normal
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# Create the model
model = ExamBehaviorCNN()

# Test with a fake exam frame
fake_frame = torch.randn(1, 3, 224, 224)  # 1 image, 3 colors, 224x224
output = model(fake_frame)
print(output)  # tensor([[ 0.23, -0.15]])
# Apply softmax to get probabilities
probs = torch.softmax(output, dim=1)
print(f"Cheating: {probs[0][0]:.1%}, Normal: {probs[0][1]:.1%}")
```

Mini Project: Cats vs Dogs Classifier

Goal: Build a CNN that looks at a photo and says "cat" or "dog."

Why this specific project? Because "cats vs dogs" transfers directly to "cheating vs normal" — same code, different labels.

```
import torch
import torch.nn as nn
```

```
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import os

# Step 1: Prepare data
# Download a cats vs dogs dataset (or use torchvision.datasets)
# Organize folders like:
#   data/train/cats/   (1000 cat images)
#   data/train/dogs/   (1000 dog images)
#   data/test/cats/    (200 cat images)
#   data/test/dogs/    (200 dog images)

transform = transforms.Compose([
    transforms.Resize((224, 224)),      # All images same size
    transforms.ToTensor(),              # Convert to tensor
    transforms.Normalize([0.485, 0.456, 0.406],  # Standard normalization
                         [0.229, 0.224, 0.225])
])

train_data = datasets.ImageFolder('data/train', transform=transform)
test_data = datasets.ImageFolder('data/test', transform=transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

print(f"Classes: {train_data.classes}")  # ['cats', 'dogs']
print(f"Training images: {len(train_data)}")
print(f"Test images: {len(test_data)}")

# Step 2: Build CNN (same structure as ExamGuard!)
class CatDogCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 2)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = CatDogCNN()
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Step 3: Train
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    model.train()
    running_loss = 0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        predictions = model(images)
        loss = loss_fn(predictions, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        correct += (predictions.argmax(1) == labels).sum().item()
        total += labels.size(0)

    print(f"Epoch {epoch+1}/10 | Loss: {running_loss/len(train_loader):.4f} | "
          f"Accuracy: {100*correct/total:.1f}%")

# Step 4: Test
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        predictions = model(images)
        correct += (predictions.argmax(1) == labels).sum().item()
        total += labels.size(0)

print(f"\nTest Accuracy: {100*correct/total:.1f}%")

# Step 5: Save the model
torch.save(model.state_dict(), 'cat_dog_cnn.pth')
```

After This Project, You Will:

- Understand how CNNs see images layer by layer
- Know how to structure image data for training

- Be able to build a CNN from scratch in PyTorch
- Be ready to swap "cats/dogs" with "cheating/normal" for ExamGuard

How This Becomes ExamGuard

Cats vs Dogs Project	ExamGuard System
Cat images	"Cheating" frames from exam cameras
Dog images	"Normal" frames from exam cameras
CatDogCNN model	ExamBehaviorCNN model
"cat" or "dog" output	"cheating" or "normal" output
Run once on a photo	Run 30 times per second on live video

The architecture is identical. The only difference is the data and the stakes.

Key Takeaway

CNN is not just another model — it is THE model that makes ExamGuard possible. Every camera frame in every exam hall passes through a CNN. Without understanding CNNs, you cannot build, debug, or improve ExamGuard. This is the single most important model in your entire project.

Image Classification — Teaching AI to Categorize What It Sees

What Is This?

Image classification is the task of training a model to look at an image and say what category it belongs to.

You do this every day without thinking:

- You see a cat → your brain says "cat"
- You see a car → your brain says "car"
- You see a student looking at their neighbor's paper → your brain says "cheating"

Image classification teaches a computer to do the same thing. You show it thousands of labeled examples, and it learns the patterns.

WHY This Matters for ExamGuard

ExamGuard's core job is classification. Every single frame from every camera needs to be classified:

```
Frame from Camera 2, Seat 15, 10:23:45 AM
        |
        v
 Image Classification Model
        |
        v
 "CHEATING" (confidence: 87%)
        |
        v
 Alert sent to supervisor
```

This is not a "nice feature." This IS ExamGuard. The entire system is an image classification pipeline running on live video.

What ExamGuard Needs to Classify:

Input	Output Categories	Confidence Needed
Student behavior frame	Normal / Suspicious / Cheating	90%+ for cheating alert
Detected object crop	Phone / Chit / Earpiece / Harmless	85%+ for object alert
Face crop	Registered student / Unknown person	95%+ for identity match
Gaze direction	Own paper / Neighbor / Room / Phone	80%+ for gaze tracking

The Complete Image Classification Pipeline

This is the exact workflow you will follow for ExamGuard:

```
Step 1: COLLECT IMAGES
  Record exam footage → Extract frames → 10,000+ frames needed

Step 2: LABEL THEM
  Human reviews each frame → Tags as "cheating" or "normal"
  This is tedious but CRITICAL — garbage labels = garbage model

Step 3: PREPROCESS
  Resize all to 224x224 → Normalize pixel values → Split 80/10/10
```

Step 4: TRAIN CNN
  Feed batches through model → Calculate loss → Update weights → Repeat

Step 5: EVALUATE
  Test on unseen images → Check accuracy, precision, recall
  Need 90%+ accuracy and LOW false positives

Step 6: DEPLOY
  Save model → Load in ExamGuard → Process live frames

Step-by-Step Breakdown

Step 1: Collect Images

For ExamGuard, you need frames from exam cameras showing different behaviors:

```
data/
 cheating/
   frame_0001.jpg   (student looking at neighbor)
   frame_0002.jpg   (student with phone under desk)
   frame_0003.jpg   (passing chit to neighbor)
   ...
   frame_5000.jpg
 normal/
   frame_0001.jpg   (student writing on own paper)
   frame_0002.jpg   (student thinking, looking up)
   frame_0003.jpg   (student reading question paper)
   ...
   frame_5000.jpg
```

How many images?

- Minimum viable: 1,000 per category (2,000 total)
- Good: 5,000 per category (10,000 total)
- Great: 10,000+ per category (20,000+ total)
- With data augmentation (next lesson): multiply all by 5x

Step 2: Label Them

```
# Images in folders ARE the labels!
# PyTorch's ImageFolder reads folder names as class labels:

from torchvision import datasets

data = datasets.ImageFolder('data/')
print(data.classes)       # ['cheating', 'normal']
print(data.class_to_idx)  # {'cheating': 0, 'normal': 1}
```

ExamGuard labeling challenge: Some frames are ambiguous. Is a student looking up "thinking" (normal) or "looking at the board to copy" (cheating)? You need clear labeling guidelines.

Step 3: Preprocess

```
from torchvision import transforms
```

```
# Every image gets the same treatment:
transform = transforms.Compose([
    transforms.Resize((224, 224)),          # Same size for all
    transforms.ToTensor(),                  # Pixels 0-255 → 0.0-1.0
    transforms.Normalize(                   # Standard normalization
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Why 224x224? Most pre-trained models expect this size. It is a good balance between detail and speed.

Split the data:

```
Total: 10,000 images
  Training:   8,000 (80%) — model learns from these
  Validation: 1,000 (10%) — tune hyperparameters
  Testing:    1,000 (10%) — final accuracy check (NEVER train on these)
```

Step 4: Train

```
from torch.utils.data import DataLoader, random_split

# Split dataset
train_size = int(0.8 * len(dataset))
val_size = int(0.1 * len(dataset))
test_size = len(dataset) - train_size - val_size

train_data, val_data, test_data = random_split(dataset, [train_size, val_size, test_size])

# Create data loaders (feeds batches to model)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)
```

Batch size 32 means the model sees 32 images at a time. Too small = slow. Too big = runs out of GPU memory.

Step 5: The Training Loop

```
import torch
import torch.nn as nn
import torch.optim as optim

model = ExamBehaviorCNN()  # From the CNN lesson
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

best_val_accuracy = 0

for epoch in range(20):
    # --- TRAINING ---
```

```
    model.train()
    train_loss = 0
    train_correct = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        predictions = model(images)
        loss = loss_fn(predictions, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        train_correct += (predictions.argmax(1) == labels).sum().item()

    train_acc = 100 * train_correct / len(train_data)

    # --- VALIDATION ---
    model.eval()
    val_correct = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            predictions = model(images)
            val_correct += (predictions.argmax(1) == labels).sum().item()

    val_acc = 100 * val_correct / len(val_data)

    print(f"Epoch {epoch+1}/20 | Train Acc: {train_acc:.1f}% | Val Acc: {val_acc:.1f}%")

    # --- SAVE BEST MODEL ---
    if val_acc > best_val_accuracy:
        best_val_accuracy = val_acc
        torch.save(model.state_dict(), 'best_exam_model.pth')
        print(f"  *** New best model saved! Val accuracy: {val_acc:.1f}% ***")
```

Step 6: Evaluate and Save

```
# Load best model
model.load_state_dict(torch.load('best_exam_model.pth'))
model.eval()

# Test on completely unseen data
test_correct = 0
total = 0
all_predictions = []
all_labels = []

with torch.no_grad():
```

```
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        predictions = model(images)

        test_correct += (predictions.argmax(1) == labels).sum().item()
        total += labels.size(0)

        all_predictions.extend(predictions.argmax(1).cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

print(f"Final Test Accuracy: {100 * test_correct / total:.1f}%")

# Detailed metrics
from sklearn.metrics import classification_report
print(classification_report(all_labels, all_predictions,
                            target_names=['cheating', 'normal']))
```

ExamGuard accuracy targets:

- Overall accuracy: 90%+
- Cheating recall: 95%+ (catch almost ALL cheaters)
- Normal precision: 95%+ (do not falsely accuse innocent students)

Common Problems and Solutions

Problem	Symptom	Solution
Overfitting	Train accuracy 99%, test accuracy 60%	Add dropout, get more data, use augmentation
Underfitting	Train accuracy 55%, test accuracy 52%	Use bigger model, train longer, lower learning rate
Class imbalance	9000 normal, 1000 cheating	Oversample cheating, use weighted loss function
Bad labels	Accuracy plateaus at 70%	Review and clean up labels

Mini Project: Exam Behavior Classifier (3 Categories)

Goal: Classify exam behavior images into: Normal, Suspicious, Cheating.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split

# Data structure:
# exam_data/
#   normal/       (student writing, reading, thinking)
#   suspicious/   (looking around briefly, stretching toward neighbor)
#   cheating/     (phone out, passing note, copying from neighbor)

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

```
])

dataset = datasets.ImageFolder('exam_data/', transform=transform)
print(f"Classes: {dataset.classes}")       # ['cheating', 'normal', 'suspicious']
print(f"Total images: {len(dataset)}")

# Split
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_data, test_data = random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

# Model — note the output is 3 classes now, not 2!
class ExamClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 14 * 14, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 3)    # 3 classes: normal, suspicious, cheating
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = ExamClassifier()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Train
for epoch in range(15):
    model.train()
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images)
```

```
        loss = loss_fn(preds, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        correct += (preds.argmax(1) == labels).sum().item()
        total += labels.size(0)

    print(f"Epoch {epoch+1}/15 | Train Accuracy: {100*correct/total:.1f}%")

# Test
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images)
        correct += (preds.argmax(1) == labels).sum().item()
        total += labels.size(0)

print(f"\nTest Accuracy: {100*correct/total:.1f}%")
torch.save(model.state_dict(), 'exam_classifier_3class.pth')
```

Why 3 Categories Instead of 2?

In real ExamGuard:

- Normal (no alert): Student writing, reading, thinking
- Suspicious (soft alert): Looking around briefly, leaning toward neighbor — supervisor gets a yellow flag
- Cheating (hard alert): Phone visible, passing notes, clearly copying — supervisor gets a red flag with evidence clip

This 3-tier system reduces false positives. A brief glance around is suspicious, not cheating. Only sustained or clear violations trigger the full alert.

Key Takeaway

Image classification is the fundamental task of ExamGuard. You collect exam frames, label them, train a CNN to classify them, and deploy it on live video. The pipeline you learn here is the exact pipeline you will use in the real system. Master this, and you have mastered ExamGuard's core intelligence.

Transfer Learning — The Game Changer for ExamGuard

What Is This?

Transfer learning means taking a model that someone else already trained on millions of images and fine-tuning it with your small dataset.

Think of it like this:

- Without transfer learning: You hire someone who has never seen a photo in their life and teach them from scratch what a person, phone, and paper look like. Takes years.
- With transfer learning: You hire someone who already knows what everything in the world looks like, and just teach them the difference between "cheating" and "normal" exam behavior. Takes hours.

This is not a shortcut. This is how the entire industry works. Google, Tesla, hospitals — everyone uses transfer learning.

WHY This Is a GAME CHANGER for ExamGuard

Here is the problem:

Training a CNN from scratch:
- Need: 1,000,000+ labeled exam images
- Time: Weeks of training on expensive GPUs
- Cost: $500-$5000 in cloud GPU rental
- Result: Maybe 80% accuracy

With transfer learning:
- Need: 5,000-10,000 labeled exam images
- Time: 2-4 hours on a laptop GPU
- Cost: $0 (your own computer)
- Result: 90-95% accuracy

You do not have millions of exam cheating images. Nobody does. But thanks to transfer learning, you do not need them. You can build a highly accurate ExamGuard model with just a few thousand images.

How Transfer Learning Works

Step 1: Start with a Pre-Trained Model

Companies like Google and Meta have trained massive models on ImageNet — a dataset of 14 million images across 20,000 categories. These models already know how to see:

What ResNet already knows (from 14 million images):
- Layer 1-5: Edges, textures, colors
- Layer 6-15: Shapes, patterns, body parts
- Layer 16-30: Objects, animals, people, scenes
- Layer 31-34: Specific categories (dog breed, car model, etc.)
- Last layer: 1000 ImageNet categories

Step 2: Remove the Last Layer

The last layer is specific to ImageNet's 1000 categories. We do not need "golden retriever" or "sports car." We need "cheating" and "normal."

Pre-trained ResNet:
[Edge detection] → [Shape detection] → [Object detection] → [1000 ImageNet classes]

↑
REMOVE THIS

Our ExamGuard model:
[Edge detection] → [Shape detection] → [Object detection] → [2 classes: cheating/normal]
↑
ADD THIS

Step 3: Freeze the Early Layers

The early layers (edges, shapes, patterns) are already perfect. Do not change them. Only train the new last layer.

Layers 1-30: FROZEN (do not change) — already know how to "see"
Layer 31-34: Fine-tune (small updates) — adapt to exam context
New last layer: TRAIN FROM SCRATCH — learn cheating vs normal

Step 4: Train on Your Data

Now train with your 5K-10K exam images. The model already understands images — it just needs to learn what cheating looks like.

The Numbers That Prove It

Approach	Training Images	Training Time	GPU Cost	Accuracy
CNN from scratch	1,000,000+	2-4 weeks	$500-5000	~80%
Transfer learning (frozen)	5,000	1-2 hours	$0	~90%
Transfer learning (fine-tuned)	10,000	2-4 hours	$0	~93-95%

Transfer learning gives you better results with 200x less data in 100x less time at zero cost.

Real ExamGuard Connection

Pre-trained ResNet50 (trained on 14 million images):
✓ Already knows what people look like
✓ Already knows what hands, heads, phones look like
✓ Already knows spatial relationships (person near table)
✓ Already understands lighting variations

What we teach it (with 5K-10K exam images):
→ "This is cheating" (student looking at neighbor's paper)
→ "This is normal" (student writing on own paper)
→ "This is suspicious" (student looking around)

Result:
ExamGuard behavior classifier — 93% accuracy — trained in 2 hours

Popular Pre-Trained Models

Model	Parameters	Speed	Accuracy	Best For
ResNet18	11M	Fast	Good	Quick prototyping
ResNet50	25M	Medium	Better	Best for ExamGuard
VGG16	138M	Slow	Good	Simple, well-understood
EfficientNet-B0	5M	Fast	Great	Mobile/edge

				deployment
MobileNetV2	3.4M	Very fast	OK	Raspberry Pi, phones

Recommendation for ExamGuard: Start with ResNet50. Good balance of accuracy and speed. Switch to EfficientNet if you need faster processing for multiple cameras.

Transfer Learning in PyTorch — Complete Code

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, random_split

# ===== Step 1: Prepare data =====
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Folder structure:
# exam_data/train/cheating/  (images)
# exam_data/train/normal/    (images)
# exam_data/test/cheating/   (images)
# exam_data/test/normal/     (images)

train_data = datasets.ImageFolder('exam_data/train', transform=transform)
test_data = datasets.ImageFolder('exam_data/test', transform=transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

print(f"Classes: {train_data.classes}")
print(f"Training: {len(train_data)} images")
print(f"Testing: {len(test_data)} images")

# ===== Step 2: Load pre-trained ResNet50 =====
model = models.resnet50(pretrained=True)  # Downloads ~100MB first time

# See what the last layer looks like:
print(model.fc)  # Linear(in_features=2048, out_features=1000)
# It outputs 1000 classes (ImageNet). We need 2 (cheating/normal).

# ===== Step 3: Freeze all layers =====
for param in model.parameters():
    param.requires_grad = False  # Do NOT update these weights

# ===== Step 4: Replace the last layer =====
model.fc = nn.Sequential(
    nn.Linear(2048, 512),
    nn.ReLU(),
```

```
    nn.Dropout(0.3),
    nn.Linear(512, 2)    # 2 classes: cheating, normal
)

# The new layer IS trainable (requires_grad=True by default)
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad)}")
# ~1 million trainable (out of 25 million total)
# The other 24 million are frozen — already trained on ImageNet

# ===== Step 5: Train only the new layer =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)  # Only optimize new layer!

best_accuracy = 0

for epoch in range(10):
    model.train()
    train_correct = 0
    train_total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        preds = model(images)
        loss = loss_fn(preds, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_correct += (preds.argmax(1) == labels).sum().item()
        train_total += labels.size(0)

    # Validate
    model.eval()
    test_correct = 0
    test_total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            preds = model(images)
            test_correct += (preds.argmax(1) == labels).sum().item()
            test_total += labels.size(0)

    train_acc = 100 * train_correct / train_total
    test_acc = 100 * test_correct / test_total
```

```
    print(f"Epoch {epoch+1}/10 | Train: {train_acc:.1f}% | Test: {test_acc:.1f}%")

    if test_acc > best_accuracy:
        best_accuracy = test_acc
        torch.save(model.state_dict(), 'examguard_resnet50.pth')
        print(f"  *** Best model saved! {test_acc:.1f}% ***")

print(f"\nBest accuracy: {best_accuracy:.1f}%")
```

Advanced: Fine-Tuning (Even Better Results)

After training the last layer, you can "unfreeze" some earlier layers and fine-tune them too:

```
# After initial training with frozen layers...

# Unfreeze the last few ResNet blocks
for param in model.layer4.parameters():
    param.requires_grad = True

# Use a VERY small learning rate for pre-trained layers
optimizer = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 0.0001},  # Small updates
    {'params': model.fc.parameters(), 'lr': 0.001}        # Normal updates
])

# Train for a few more epochs
# This typically adds 2-3% accuracy improvement
```

When to fine-tune:

- Your data is somewhat different from ImageNet (exam scenes vs generic photos)
- You have enough data (5K+) to avoid overfitting
- The frozen model accuracy plateaus and you need more

Mini Project: Custom Image Classifier with ResNet

Goal: Use pre-trained ResNet to classify your own custom images — any categories you choose.

Suggested approach:

- 1. Pick 3-5 categories (e.g., "sitting normally", "leaning over", "using phone", "looking sideways")
- 2. Collect 100-200 images per category (take photos yourself or find online)
- 3. Organize into folders
- 4. Apply transfer learning
- 5. Test on new images

```
# After training, use the model on a single new image:

from PIL import Image

def predict_single_image(model, image_path, class_names):
    model.eval()

    image = Image.open(image_path)
```

```
    image = transform(image).unsqueeze(0).to(device)  # Add batch dimension

    with torch.no_grad():
        output = model(image)
        probs = torch.softmax(output, dim=1)
        confidence, predicted = torch.max(probs, 1)

    print(f"Prediction: {class_names[predicted.item()]}")
    print(f"Confidence: {confidence.item():.1%}")

# Test it
predict_single_image(model, 'test_frame.jpg', ['cheating', 'normal'])
# Output: Prediction: cheating
#         Confidence: 91.3%
```

Key Takeaway

Transfer learning is the reason ExamGuard is buildable by a small team. Without it, you would need millions of images and weeks of training. With it, you need thousands of images and hours of training. This is not optional — it is the practical strategy that makes the entire project feasible. Every serious computer vision project in the real world uses transfer learning.

Data Augmentation — Multiply Your Training Data for Free

What Is This?

Data augmentation means taking your existing training images and creating variations of them — flipped, rotated, brighter, darker, cropped differently — so your model sees more examples without you collecting more data.

Think of it like this:

- You have a photo of a student cheating
- You flip it horizontally → now you have TWO training images
- You rotate it 5 degrees → now you have THREE
- You make it brighter → FOUR
- You make it darker → FIVE
- You crop slightly differently → SIX

One image became six. Do this to all 10,000 images and you have 60,000 training examples.

WHY This Is Critical for ExamGuard

The Problem

You have a limited dataset. Maybe 5,000-10,000 labeled exam frames. That is not enough to handle every real-world condition:

```
Conditions your model will face in real exams:
- Exam Hall A: Bright fluorescent lights
- Exam Hall B: Dim natural light from windows
- Exam Hall C: Mixed lighting, some shadows
- Morning exams: Warm sunlight from east windows
- Evening exams: Cool artificial light
- Camera 1: Slightly tilted angle
- Camera 3: Students partially occluded by other students
```

If you only train on images from Hall A with bright lights, your model fails in Hall B with dim lights. It has never seen dim lighting.

The Solution: Augmentation

```
Original 10,000 images
  |
  v
Flip horizontally     → +10,000 = 20,000
Rotate ±10 degrees    → +10,000 = 30,000
Brightness variations → +10,000 = 40,000
Add slight noise      → +10,000 = 50,000
  |
  v
50,000 training images — 5x more data, ZERO additional collection!
```

Now your model has seen bright images, dark images, rotated images, and noisy images. It works in ALL exam halls.

Augmentation Techniques for ExamGuard

1. Horizontal Flip

Original: Flipped:
Student looks RIGHT → ← Student looks LEFT

Why it works: Cheating looks the same whether the student looks left or right. Doubles your data instantly.

ExamGuard use: A student looking at their left neighbor is the same behavior as looking at their right neighbor.

2. Small Rotation (±5 to ±15 degrees)

Original: Rotated +10°: Rotated -10°:

```
 |          /          \
 |         /            \
 |        /              \
```

Why it works: Cameras are not always perfectly straight. Small rotations simulate real camera angles.

ExamGuard use: Different cameras are mounted at slightly different angles.

3. Brightness/Contrast Changes

Original: Brighter: Darker: More contrast:
[normal] [washed out] [shadowy] [sharp edges]

Why it works: Different exam halls have different lighting. Morning light differs from evening light.

ExamGuard use: Model must work at 8 AM (sunlight) and 5 PM (artificial light) equally well.

4. Random Crop / Zoom

Original (full frame): Cropped (zoomed in):
[student at desk] [student at d]

Why it works: Students sit at different distances from cameras. Some are close, some are far.

ExamGuard use: Row 1 students appear large in frame, Row 10 students appear small.

5. Adding Noise

Original: With noise:
[clean image] [slightly grainy image]

Why it works: Cheap cameras produce noisy images. Night/low-light adds grain.

ExamGuard use: Not all exam halls have high-quality cameras.

6. Color Jitter

Original: Warmer: Cooler:
[neutral] [yellowish] [bluish]

Why it works: Different light sources have different color temperatures.

ExamGuard use: Fluorescent lights (cool/blue) vs incandescent lights (warm/yellow).

What NOT to Augment

Some augmentations would create unrealistic images:

Augmentation	Use it?	Why

Horizontal flip	YES	Cheating looks same in mirror
Vertical flip	NO	Students are never upside down
180° rotation	NO	Unrealistic for exam setting
Extreme zoom	NO	Loses important context
Color inversion	NO	Creates alien-looking images

Rule: Only augment in ways that produce images that could realistically appear in an exam hall.

Implementation in PyTorch

Training Transforms (WITH augmentation)

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.Resize((256, 256)),            # Slightly larger than needed
    transforms.RandomCrop(224),               # Random crop to 224x224
    transforms.RandomHorizontalFlip(p=0.5),   # 50% chance of flipping
    transforms.RandomRotation(10),            # ±10 degree rotation
    transforms.ColorJitter(
        brightness=0.3,                       # ±30% brightness change
        contrast=0.3,                         # ±30% contrast change
        saturation=0.2,                       # ±20% color intensity
        hue=0.1                               # ±10% color shift
    ),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 2.0)),  # Slight blur
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Test Transforms (NO augmentation — we want clean evaluation)

```
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),            # Just resize, nothing else
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Important: NEVER augment test data. Test data must represent real-world conditions accurately so your accuracy numbers are honest.

Using Both in DataLoader

```
from torchvision import datasets
from torch.utils.data import DataLoader

train_data = datasets.ImageFolder('exam_data/train', transform=train_transform)
test_data = datasets.ImageFolder('exam_data/test', transform=test_transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

# Each time an image is loaded for training, a RANDOM augmentation is applied
# So the model sees a different version of each image every epoch!
# Over 20 epochs, each image appears in ~20 different variations
```

ExamGuard Augmentation Strategy

```
# ExamGuard-specific augmentation pipeline
examguard_train_transform = transforms.Compose([
    # Size handling
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),

    # Simulate different camera orientations
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=8),

    # Simulate different lighting conditions
    transforms.ColorJitter(
        brightness=0.4,      # Exam halls vary a LOT in lighting
        contrast=0.3,
        saturation=0.2,
        hue=0.05
    ),

    # Simulate camera quality variation
    transforms.RandomChoice([
        transforms.GaussianBlur(3, sigma=(0.1, 1.5)),   # Cheap camera blur
        transforms.Lambda(lambda x: x),                 # No blur (good camera)
    ]),

    # Standard preprocessing
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),

    # Simulate partial occlusion (another student blocking view)
    transforms.RandomErasing(p=0.2, scale=(0.02, 0.15)),
])
```

RandomErasing is especially useful for ExamGuard — it randomly blacks out a small rectangle in the image, simulating when one student partially blocks the camera's view of another student.

The Impact: Real Numbers

Here is what augmentation does to ExamGuard model performance:

```
Experiment: ExamGuard behavior classification

WITHOUT augmentation:
  Training data: 5,000 images
  Training accuracy: 98% (overfitting!)
  Test accuracy: 76%
  Real-world accuracy: ~70% (fails in different lighting)

WITH augmentation:
  Training data: 5,000 images (augmented to ~25,000 effective)
  Training accuracy: 91%
```

Test accuracy: 89%
Real-world accuracy: ~87% (works across halls and lighting)

The gap between training and test accuracy shrank from 22% to 2%. That means the model generalizes — it works on images it has never seen before.

Mini Project: Augmentation Impact Comparison

Goal: Take 100 images, train a model without augmentation, then with augmentation, and compare results.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, Subset
import random

# ===== Setup =====
# Use a small subset of data to clearly show the augmentation effect
# Download: A small image dataset (e.g., CIFAR-10 or your own 100 images)

# ===== Experiment 1: NO augmentation =====
print("=" * 50)
print("EXPERIMENT 1: Without Data Augmentation")
print("=" * 50)

simple_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_data_simple = datasets.ImageFolder('mini_data/train', transform=simple_transform)
test_data_simple = datasets.ImageFolder('mini_data/test', transform=simple_transform)

train_loader_simple = DataLoader(train_data_simple, batch_size=16, shuffle=True)
test_loader_simple = DataLoader(test_data_simple, batch_size=16)

# Use a small pre-trained model
model1 = models.resnet18(pretrained=True)
for param in model1.parameters():
    param.requires_grad = False
model1.fc = nn.Linear(512, len(train_data_simple.classes))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model1 = model1.to(device)
optimizer1 = optim.Adam(model1.fc.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(10):
    model1.train()
    for images, labels in train_loader_simple:
```

```
        images, labels = images.to(device), labels.to(device)
        loss = loss_fn(model1(images), labels)
        optimizer1.zero_grad()
        loss.backward()
        optimizer1.step()

model1.eval()
correct = sum(
    (model1(img.to(device)).argmax(1) == lab.to(device)).sum().item()
    for img, lab in test_loader_simple
)
total = len(test_data_simple)
print(f"Test Accuracy WITHOUT augmentation: {100*correct/total:.1f}%")

# ===== Experiment 2: WITH augmentation =====
print("\n" + "=" * 50)
print("EXPERIMENT 2: With Data Augmentation")
print("=" * 50)

augmented_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_data_aug = datasets.ImageFolder('mini_data/train', transform=augmented_transform)
test_data_aug = datasets.ImageFolder('mini_data/test', transform=simple_transform)  # NO aug for test!

train_loader_aug = DataLoader(train_data_aug, batch_size=16, shuffle=True)
test_loader_aug = DataLoader(test_data_aug, batch_size=16)

model2 = models.resnet18(pretrained=True)
for param in model2.parameters():
    param.requires_grad = False
model2.fc = nn.Linear(512, len(train_data_aug.classes))
model2 = model2.to(device)
optimizer2 = optim.Adam(model2.fc.parameters(), lr=0.001)

for epoch in range(10):
    model2.train()
    for images, labels in train_loader_aug:
        images, labels = images.to(device), labels.to(device)
        loss = loss_fn(model2(images), labels)
        optimizer2.zero_grad()
        loss.backward()
        optimizer2.step()
```

```
model2.eval()
correct = sum(
   (model2(img.to(device)).argmax(1) == lab.to(device)).sum().item()
   for img, lab in test_loader_aug
)
total = len(test_data_aug)
print(f"Test Accuracy WITH augmentation: {100*correct/total:.1f}%")

# ===== Compare =====
print("\n" + "=" * 50)
print("COMPARISON")
print("=" * 50)
print("You should see 5-15% improvement with augmentation,")
print("especially when training data is small (100 images).")
print("The smaller your dataset, the bigger the augmentation impact.")
```

What You Will Learn:

- How dramatically augmentation improves accuracy on small datasets
- That test data should NEVER be augmented
- Which augmentations make sense for your use case
- How to build augmentation pipelines in PyTorch

Key Takeaway

Data augmentation is free data. It costs nothing, takes no extra collection time, and can boost your model accuracy by 5-15%. For ExamGuard, where you will never have millions of labeled exam images, augmentation is not optional — it is essential. It is the difference between a model that works only in the lab and one that works in every exam hall, under every lighting condition, with every camera angle.

Phase 4: Deep Learning — Learning Resources

YouTube Tutorials (Free)

PyTorch Fundamentals

- "PyTorch for Deep Learning & Machine Learning — Full Course" by freeCodeCamp
- 25+ hours, covers everything from tensors to deployment
- https://www.youtube.com/watch?v=V_xro1bcAuQ
- "PyTorch Tutorials" by Sentdex
- Practical, project-based approach
- Great for people who learn by doing
- "PyTorch in 100 Seconds" by Fireship
- Quick overview before diving deep

CNN (Convolutional Neural Networks)

- "But what is a convolution?" by 3Blue1Brown
- Beautiful visual explanation of convolutions
- Must-watch before coding anything
- "CNN Explainer" by Poloclub (interactive website)
- https://poloclub.github.io/cnn-explainer/
- See CNN layers activate in real time on actual images
- "Convolutional Neural Networks Explained" by deeplizard
- Step-by-step, beginner-friendly series
- "CS231n: Convolutional Neural Networks for Visual Recognition" by Stanford (YouTube)
- University-level but incredibly clear lectures by Andrej Karpathy

Transfer Learning

- "Transfer Learning with PyTorch" by PyTorch official
- Exact workflow you will use for ExamGuard
- "Transfer Learning Explained" by StatQuest
- Josh Starmer breaks it down clearly with no jargon
- "Fine-tuning pre-trained models in PyTorch" — search for latest tutorials
- New tutorials appear regularly as models improve

Data Augmentation

- "Data Augmentation Techniques for Deep Learning" — search YouTube
- Many short tutorials showing before/after effects
- "Albumentations library tutorial"
- Advanced augmentation library (beyond torchvision transforms)
- Very popular in Kaggle competitions

Courses (Structured Learning)

Highly Recommended (Free)

- fast.ai — Practical Deep Learning for Coders
- https://course.fast.ai/
- FREE, 7 lessons, top-down approach (build first, theory later)
- Created by Jeremy Howard (former Kaggle president)
- Teaches PyTorch through the fastai library
- You will build image classifiers in Lesson 1
- This is the single best free deep learning course available

Recommended (Paid but worth it)

- DeepLearning.AI — Deep Learning Specialization (Coursera)
- By Andrew Ng (co-founder of Google Brain)
- 5 courses covering everything from basics to advanced
- Uses TensorFlow, but concepts apply to PyTorch
- Free to audit, ~$49/month for certificates
- Financial aid available
- DeepLearning.AI — TensorFlow Developer Professional Certificate (Coursera)
- More practical, project-focused
- Covers CNNs, transfer learning, and image classification specifically

Practice Platforms

Kaggle (Free)

- "Dogs vs Cats" competition — perfect first image classification project
- https://www.kaggle.com/c/dogs-vs-cats
- "Digit Recognizer" (MNIST) — beginner-friendly
- https://www.kaggle.com/c/digit-recognizer
- "Intel Image Classification" — classify scenes (buildings, forest, etc.)
- Good transfer learning practice
- Kaggle Learn: Intro to Deep Learning — free micro-course
- https://www.kaggle.com/learn/intro-to-deep-learning

Google Colab (Free GPU)

- https://colab.research.google.com/
- Free GPU access for training models
- No setup needed — runs in your browser
- Perfect for following along with tutorials

Books (Optional, for deeper understanding)

- "Dive into Deep Learning" (Free online)
- https://d2l.ai/

- Interactive book with runnable code
- Available in PyTorch, TensorFlow, and JAX versions
- "Deep Learning with Python" by Francois Chollet
- Written by the creator of Keras
- Very practical, lots of code examples

GitHub Repositories

- PyTorch official examples: https://github.com/pytorch/examples
- Image classification, transfer learning, and more
- PyTorch image models (timm): https://github.com/huggingface/pytorch-image-models
- Hundreds of pre-trained models for transfer learning
- Albumentations: https://github.com/albumentations-team/albumentations
- Advanced data augmentation library

Recommended Learning Order

Week 1-2: PyTorch basics
→ Watch: freeCodeCamp PyTorch course (first 5 hours)
→ Do: MNIST digit classifier mini project
→ Practice: Kaggle Digit Recognizer

Week 3-4: CNNs
→ Watch: 3Blue1Brown convolution video
→ Watch: deeplizard CNN series
→ Do: Cats vs Dogs CNN mini project
→ Play with: CNN Explainer website

Week 5-6: Image Classification + Transfer Learning
→ Start: fast.ai course (Lessons 1-3)
→ Do: Transfer learning mini project with ResNet
→ Practice: Kaggle Dogs vs Cats competition

Week 7: Data Augmentation
→ Watch: Augmentation tutorials
→ Do: Augmentation comparison mini project
→ Experiment: Try different augmentations on your dataset

Week 8: Put it all together
→ Build: Full pipeline (data → augmentation → transfer learning → evaluation)
→ Target: 90%+ accuracy on a custom image classification task

Quick Reference: What to Search When Stuck

Problem	Search This
PyTorch installation issues	"install pytorch [your OS] [cuda version]"
Model not learning	"pytorch model not training troubleshooting"
Out of GPU memory	"pytorch reduce GPU memory usage"
Overfitting	"pytorch prevent overfitting CNN"
Low accuracy	"improve CNN accuracy pytorch"
Confusion about tensors	"pytorch tensor tutorial beginner"

Transfer learning not working	"pytorch transfer learning fine-tuning tips"

Key Tip

Do not try to learn everything before building. Follow the fast.ai philosophy: build something first, then understand why it works. You will learn 10x faster by training a real model and seeing what breaks than by reading theory for weeks.